

Josetech

Plataforma de Reparación de Dispositivos
Electrónicos

DOCUMENTACIÓN TÉCNICA DEL PROYECTO

Autor: José (Joselillo412)

Repositorio: github.com/Joselillo412/proyectoJosetech

Versión: 1.0 | Mayo 2026



ÍNDICE

ÍNDICE.....	2
1. Introducción.....	3
2. Características principales.....	4
2.1 Para Usuarios	4
2.2 Para Guías / Técnicos	4
2.3 Para Administradores	4
3. Tecnologías utilizadas.....	6
3.1 Frontend.....	6
3.2 Backend y gestión de datos	8
3.3 Servicios Externos	10
API de iFixit	10
API de Icecat	10
Sistema de correo electrónico (Laravel Mail)	10
4. Arquitectura del Proyecto.....	11
4.1 Esquema de archivos	11
4.2 Diagrama entidad-relación	12
4.3 Diagrama de clases del modelo	14
4.4 Casos de Uso	16
4.5 Endpoints principales	17
4.5.1 Dispositivos y Catálogo	17
4.5.2 Autenticación	17
4.5.3 Usuarios, Pedidos y Contabilidad	17
5. Instalación y configuración básica.....	19
5.1 Prerrequisitos	19
5.2 Configuración del Frontend	19
5.3 Configuración del Backend	19
6. Conclusiones.....	21
6.1 Dificultades	21
6.2 Mejoras futuras	23
7. Bibliografía.....	24

1. Introducción

Josetech es una aplicación web completa orientada a la gestión integral de reparaciones de dispositivos electrónicos, desarrollada como proyecto de fin de ciclo. La plataforma cubre el ciclo de vida completo de una reparación: desde que el cliente solicita el servicio hasta que el administrador registra el cobro en contabilidad.

El proyecto nació con el objetivo de digitalizar y modernizar el proceso habitual de un taller de reparación, eliminando el papel y los registros manuales. Gracias a su arquitectura cliente-servidor desacoplada, Josetech puede evolucionar de forma independiente tanto en la capa de presentación como en la lógica de negocio.

La aplicación se estructura en dos grandes bloques:

- **Frontend:** Desarrollado con Vue 3 + Vite + SCSS, proporciona una interfaz moderna, reactiva y completamente responsiva para clientes, técnicos y administradores.
- **Backend:** Construido sobre Laravel 12 con API RESTful autenticada mediante Laravel Sanctum, gestiona todos los datos, reglas de negocio y comunicaciones con servicios externos.

Públicamente, Josetech ofrece un amplio catálogo de dispositivos compatibles organizados por marca y tipo (móviles, tablets, consolas) con más de 30 fabricantes soportados, un sistema de presupuesto online, una galería de trabajos realizados y un formulario de consulta. Para los usuarios registrados, permite solicitar reparaciones con seguimiento en tiempo real mediante un código único. El panel de administración ofrece control total sobre pedidos, usuarios, dispositivos y contabilidad.

Desde el punto de vista técnico, el proyecto integra dos APIs externas: iFixit para obtener la dificultad de reparación de cada modelo, e Icecat para enriquecer las fichas de producto con especificaciones técnicas oficiales. El sistema de correo electrónico de Laravel notifica automáticamente al administrador y al cliente cuando se genera un nuevo pedido.

Este documento recoge la arquitectura, tecnologías, estructura de datos, endpoints y guía de instalación del proyecto, sirviendo como referencia técnica completa tanto para desarrolladores que quieran contribuir como para evaluadores del proyecto.

2. Características principales

Josetech está diseñada con tres perfiles de usuario claramente diferenciados, cada uno con su propio conjunto de funcionalidades:

2.1 Para Usuarios

- Registro y autenticación con token Bearer (Laravel Sanctum), con protección de contraseñas mediante Bcrypt.
- Catálogo interactivo de dispositivos compatible con más de 30 marcas: Apple, Samsung, Xiaomi, Google, OnePlus, Huawei, Honor, Nothing, Motorola, Poco, Vivo, Oppo, Realme y más, organizadas por tipo (móviles, tablets y consolas).
- Solicitud de reparación online desde la vista autenticada /solicitar-reparacion, eligiendo dispositivo, tipo de avería y descripción.
- Consulta del estado de un pedido mediante código de seguimiento único (formato JT-XXXXXX) sin necesidad de estar registrado.
- Panel de perfil personal con historial de pedidos y posibilidad de actualizar los datos de cuenta.
- Solicitud de recuperación de contraseña que notifica al administrador para restablecerla manualmente.
- Galería pública de trabajos realizados y página de consulta/contacto.

2.2 Para Guías / Técnicos

- El sistema integra la API pública de iFixit para obtener la dificultad estimada de reparación de cualquier modelo registrado, ayudando al técnico a evaluar el trabajo antes de aceptarlo.
- Cada pedido incluye descripción del problema, tipo de reparación, precio estimado y estado actualizable.
- Notificación automática por correo electrónico cuando se recibe un nuevo pedido (PedidoAdminMail y PedidoClienteMail).

2.3 Para Administradores

- Panel de administración completo en /admin, protegido por guardia de rol en el router de Vue y verificado también en el backend.
- Gestión completa de pedidos: listado con información de cliente y dispositivo, cambio de estado (Pendiente, En proceso, Completado, Entregado...).
- Gestión de usuarios: listado, cambio de rol (cliente/admin) y restablecimiento de contraseña.
- CRUD completo de dispositivos del catálogo y de servicios ofertados con precio, icono y categoría.
- Módulo de contabilidad: registro de ingresos vinculados a pedidos, piezas cambiadas, costes, ganancias y método de pago.

3. Tecnologías utilizadas

3.1 Frontend

El frontend de Josetech es una Single Page Application (SPA) construida con el ecosistema moderno de Vue.js. A continuación se detallan las principales tecnologías y dependencias:

Tecnología	Versión	Descripción
Vue.js 3	^3.5.29	Framework reactivo de JavaScript para la construcción de interfaces de usuario basado en Composition API.
Vite	^7.3.1	Herramienta de desarrollo y bundler ultrarrápido que ofrece Hot Module Replacement (HMR) en desarrollo y builds optimizados para producción.
Vue Router	^5.0.3	Enrutador oficial para Vue.js. Gestiona la navegación entre vistas, las rutas protegidas (requiresAuth, requiresAdmin) y la carga diferida (lazy loading) de componentes.
Pinia	^3.0.4	Store de estado global oficial para Vue 3. Gestiona la sesión del usuario (auth.js), permitiendo que cualquier componente acceda al perfil y rol del usuario autenticado.
SASS Embedded	^1.98.0	Preprocesador CSS para los estilos globales base (base.css, main.css).
Prettier	3.8.1	Formateador de código automático que garantiza un estilo consistente en todos los archivos del proyecto frontend.

La estructura de vistas del frontend es especialmente extensa, con más de 50 vistas organizadas en las siguientes categorías principales:

- Vistas de autenticación: LoginView, RegisterView, ForgotPasswordView, ProfileView.
- Catálogo de móviles (14 marcas): Apple, Samsung, Xiaomi, Google, OnePlus, Huawei, Honor, Nothing, Motorola, Poco, Vivo, Oppo, Realme.
- Catálogo de tablets (13 marcas): Apple iPad, Samsung, Xiaomi, Google, OnePlus, Huawei, Honor, Lenovo, Motorola, Poco, Vivo, Oppo, Realme.
- Catálogo de consolas (3 plataformas): Sony PlayStation, Microsoft Xbox, Nintendo.

- Vistas funcionales: HomeView, CatalogView, GaleriaView, ConsultaView, SolicitarReparacionView, PresupuestoView.
- Panel de administración: AdminPanelView con control total del sistema.

3.2 Backend y gestión de datos

El backend está desarrollado con Laravel 12, el framework PHP más popular. Expone una API RESTful que consume el frontend Vue y cualquier cliente compatible con HTTP.

Tecnología	Versión	Descripción
Laravel	^12.0	Framework PHP de arquitectura MVC. Proporciona el sistema de rutas, controladores, modelos Eloquent ORM, migraciones, seeders, colas de trabajo y sistema de correo.
PHP	^8.2	Lenguaje de programación del lado del servidor. La versión 8.2 aporta mejoras de rendimiento y soporte a tipos de intersección y propiedades readonly.
Laravel Sanctum	^4.0	Sistema de autenticación para SPAs y APIs mediante tokens de acceso personal (Bearer Token). Gestiona la creación, validación y revocación de tokens.
Eloquent ORM	Integrado	ORM activo de Laravel que mapea los modelos PHP a tablas de base de datos. Permite definir relaciones (hasMany, belongsTo, etc.) y ejecutar consultas de forma expresiva.
MySQL / SQLite	Configurable	Base de datos relacional. En desarrollo se puede usar SQLite para mayor comodidad; en producción se recomienda MySQL.
Laravel Migrations	Integrado	Sistema de control de versiones para la base de datos. Permite crear, modificar y revertir la estructura de tablas de forma programática.
Laravel Mail	Integrado	Sistema de correo que usa clases Mailable (PedidoAdminMail, PedidoClienteMail) para enviar notificaciones HTML por email cuando se registra un nuevo pedido.
Laravel Tinker	^2.10.1	REPL interactivo de consola para depurar y probar código directamente contra la base de datos en tiempo real.

Los modelos principales y sus relaciones son:

- User: tiene muchos Pedido (hasMany).
- Dispositivo: referenciado por Pedido y Averia.
- Pedido: pertenece a User y a Dispositivo. Genera automáticamente su código_seguimiento (JT-XXXXXX) en el evento creating.

- **Averia:** pertenece a Dispositivo, tiene muchos LineaPedido.
- **LineaPedido:** pertenece a Pedido y a Averia, almacena el precio_aplicado en el momento del pedido.
- **Servicio:** entidad independiente con nombre, precio, icono, categoría y nota extra para el catálogo de servicios.
- **Contabilidad:** registro financiero vinculado a un Pedido con coste de piezas, ganancias, total cobrado y método de pago.

3.3 Servicios Externos

Josetech integra dos APIs externas para enriquecer la información disponible en la plataforma:

API de iFixit

iFixit es la mayor plataforma colaborativa de guías de reparación del mundo. Josetech consume su API pública (v2.0) para obtener la dificultad de reparación estimada de cualquier modelo de dispositivo.

- Endpoint utilizado:
`https://www.ifixit.com/api/2.0/search/{modelo}?filter=guide`
- Información extraída: el campo `difficulty` del primer resultado (Very Easy, Easy, Moderate, Difficult, Very Difficult).
- Gestión de errores: si el modelo no se encuentra o la petición falla, se devuelve el valor por defecto Desconocida.
- Implementación: clase `IFixitService` en `app/Services/Api/IFixitService.php`.

API de Icecat

Icecat es el mayor catálogo mundial de especificaciones técnicas de productos electrónicos. Josetech usa su API pública (`openicecat-live`) para enriquecer las fichas de portátiles y dispositivos con información oficial del fabricante.

- Endpoint utilizado:
`https://live.icecat.biz/api/?UserName=openicecat-live&Language=es&Brand={marca}&ProductCode={modelo}`
- Información extraída: nombre oficial del producto, fecha de lanzamiento y descripción larga.
- Gestión de errores: si el producto no se encuentra (`msg != OK`) o la petición falla, se devuelve `null`.
- Implementación: clase `IcecatService` en `app/Services/Api/IcecatService.php`.

Sistema de correo electrónico (Laravel Mail)

Josetech utiliza el sistema de correo nativo de Laravel para enviar notificaciones automáticas:

- `PedidoAdminMail`: notifica al administrador del taller cuando se registra una nueva reparación, con el asunto `NUEVA REPARACION SOLICITADA: #JT-XXXXXX`.
- `PedidoClienteMail`: notifica al cliente confirmando la recepción de su solicitud.
- Las plantillas de correo se ubican en `resources/views/emails/`.

4. Arquitectura del Proyecto

4.1 Esquema de archivos

La estructura del repositorio se divide en tres carpetas principales:

```
proyectoJosetech/
  backend/                                # API Laravel 12
    app/
      Http/Controllers/Api/              # AuthController, AdminController,
                                          # DispositivoController, PedidoController
      Http/Middleware/                   # CheckAdmin.php
      Mail/                              # PedidoAdminMail, PedidoClienteMail
      Models/                           # User, Pedido, Dispositivo, Averia,
                                          # LineaPedido, Servicio, Contabilidad
      Services/Api/                      # IFixitService, IcecatService
    database/
      migrations/                        # 9 ficheros de migración
      seeders/                          # DatabaseSeeder, DispositivosApiSeeder,
                                          # ServicioSeeder
    routes/
      api.php                           # Todas las rutas REST (publicas + protegidas)
      resources/views/emails/           # Plantillas HTML de correo

  frontend/                              # SPA Vue 3 + Vite
    src/
      views/                            # +50 vistas (.vue)
      components/                       # Header.vue y componentes de iconos
      router/index.js                   # Rutas SPA con guardas de navegacion
      stores/auth.js                   # Store Pinia para sesion de usuario
      assets/                           # CSS global (Tailwind base)
    index.html                          # Punto de entrada HTML
    vite.config.js                      # Configuracion de Vite
    tailwind.config.js                  # Configuracion de Tailwind

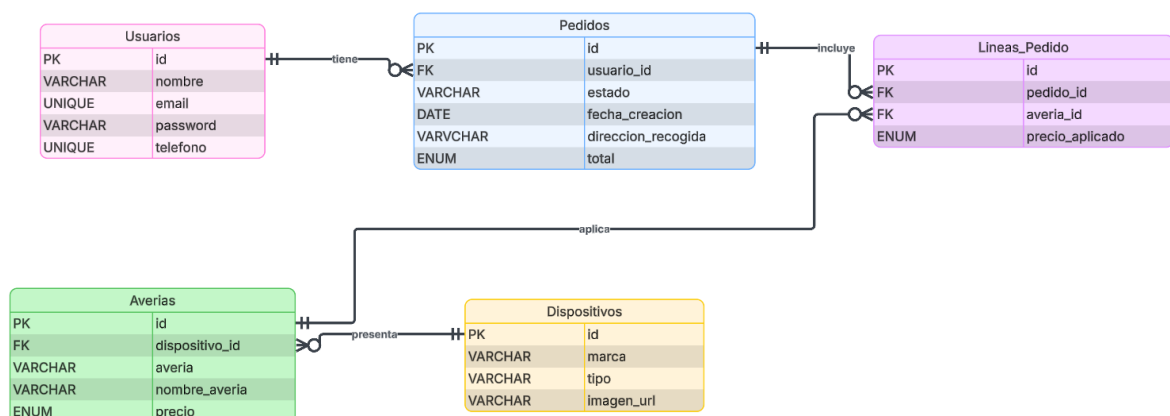
  Documentacion/                        # PDFs y diagramas del proyecto
```

La separación entre frontend y backend permite desplegarlos de forma independiente. El frontend puede alojarse en cualquier CDN o servidor de archivos estáticos (Vercel, Netlify, etc.), mientras el backend puede desplegarse en cualquier servidor con PHP 8.2+ (Forge, Heroku, VPS, etc.).

4.2 Diagrama entidad-relación

El modelo de datos relacional de Josetech está compuesto por 7 tablas principales. A continuación se describe cada entidad y sus relaciones:

Tabla	Campos principales	Relaciones
users	id, nombre, email, telefono, password, rol (cliente admin)	1:N con pedidos
dispositivos	id, marca, modelo, tipo, imagen_url	1:N con averias y pedidos
averias	id, dispositivo_id (FK), nombre, precio	N:1 dispositivos; 1:N linea_pedidos
pedidos	id, user_id (FK), dispositivo_id (FK), codigo_seguimiento, tipo_reparacion, descripcion, precio_estimado, estado	N:1 users, N:1 dispositivos; 1:N linea_pedidos y contabilidad
linea_pedidos	id, pedido_id (FK), averia_id (FK), precio_aplicado	N:1 pedidos; N:1 averias
servicios	id, nombre, precio, icono, extra_nota, categoria	Entidad independiente
contabilidad	id, pedido_id (FK), piezas_cambiadas, coste_piezas, ganancias, total_cobrado, metodo_pago	N:1 pedidos



Descripción del diagrama E-R:

- Un Usuario puede hacer muchos Pedidos a lo largo del tiempo.

- Cada Pedido está asociado a un Dispositivo concreto (el equipo a reparar).
- Un Dispositivo puede tener varias Averias definidas con sus precios estándar.
- Un Pedido puede contener varias LineaPedido (averias seleccionadas), cada una con el precio que se aplicó en ese momento (precio_aplicado), desacoplado del precio estándar de la avería para mantener el histórico.
- La tabla Contabilidad registra el cierre económico de un Pedido: piezas usadas, costes reales, ganancias netas, total cobrado y método de pago.
- La tabla Servicios es independiente y representa el catálogo de servicios genéricos ofrecidos por el taller (pantallas, baterías, limpiezas, etc.) mostrados en la página principal.

4.3 Diagrama de clases del modelo

Los modelos Eloquent de Laravel representan directamente la capa de dominio de la aplicación. A continuación se detalla la estructura de clases:

Clase User

- Traits: HasApiTokens, HasFactory, Notifiable.
- Fillable: nombre, email, telefono, password, rol.
- Hidden: password, remember_token.
- Casts: email_verified_at (datetime), password (hashed).
- Relaciones: pedidos() -> hasMany(Pedido::class).

Clase Dispositivo

- Traits: HasFactory.
- Fillable: marca, modelo, tipo, imagen_url.
- Sin relaciones declaradas explícitamente (es referenciado por Pedido y Averia).

Clase Averia

- Traits: HasFactory.
- Fillable: dispositivo_id, nombre, precio.
- Relaciones: dispositivo() -> belongsTo(Dispositivo::class); lineaPedidos() -> hasMany(LineaPedido::class).

Clase Pedido

- Traits: HasFactory.
- Fillable: user_id, dispositivo_id, codigo_seguimiento, tipo_reparacion, descripcion, precio_estimado, estado.
- Boot hook: genera automaticamente el codigo_seguimiento con el prefijo JT- y 6 caracteres aleatorios en mayúsculas.
- Relaciones: usuario() -> belongsTo(User::class, user_id); dispositivo() -> belongsTo(Dispositivo::class, dispositivo_id).

Clase LineaPedido

- Traits: HasFactory.
- Fillable: pedido_id, averia_id, precio_aplicado.
- Relaciones: pedido() -> belongsTo(Pedido::class); averia() -> belongsTo(Averia::class).

Clase Servicio

- Fillable: nombre, precio, icono, extra_nota, categoria.

Clase Contabilidad

- Traits: HasFactory.
- Table: contabilidad (nombre de tabla personalizado).
- Fillable: pedido_id, piezas_cambiadas, coste_piezas, ganancias, total_cobrado, metodo_pago.

4.4 Casos de Uso

Los principales casos de uso de la aplicación se agrupan por actor:

Actor: Usuario Anónimo

- Navegar por el catálogo de dispositivos (móviles, tablets, consolas) sin registrarse.
- Ver el listado de servicios y precios en la página principal.
- Ver la galería de trabajos realizados.
- Consultar el estado de un pedido por su código de seguimiento en /consulta.
- Registrarse como nuevo cliente.
- Iniciar sesión con email y contraseña.
- Solicitar recuperación de contraseña.

Actor: Cliente Autenticado

- Todas las acciones del usuario anónimo.
- Solicitar una nueva reparación seleccionando dispositivo, tipo de avería y descripción.
- Consultar su historial de pedidos desde el panel de perfil.
- Actualizar su información de perfil (nombre, email, teléfono).
- Cerrar sesión (revoca el token Sanctum).

Actor: Administrador

- Todas las acciones del cliente autenticado.
- Acceder al panel de administración en /admin.
- Listar, filtrar y actualizar el estado de todos los pedidos del sistema.
- Gestionar usuarios: listar, cambiar rol, restablecer contraseña.
- CRUD completo de dispositivos del catálogo.
- CRUD completo del catálogo de servicios ofertados.
- Registrar y consultar entradas de contabilidad vinculadas a pedidos.

4.5 Endpoints principales

La API REST del backend está disponible bajo el prefijo /api. A continuación se listan los endpoints organizados por módulo:

4.5.1 Dispositivos y Catálogo

Método	Ruta	Descripción
GET	/api/dispositivos	Lista todos los dispositivos del catálogo (público).
GET	/api/dispositivos/{id}	Detalle de un dispositivo concreto (público).
GET	/api/servicios	Lista todos los servicios disponibles (público).
POST	/api/admin/dispositivos	Crea un nuevo dispositivo en el catálogo (admin).
PUT	/api/admin/dispositivos/{id}	Actualiza un dispositivo existente (admin).
DELETE	/api/admin/dispositivos/{id}	Elimina un dispositivo del catálogo (admin).
POST	/api/admin/servicios	Crea un nuevo servicio (admin).
PUT	/api/admin/servicios/{id}	Actualiza un servicio (admin).
DELETE	/api/admin/servicios/{id}	Elimina un servicio (admin).

4.5.2 Autenticación

Método	Ruta	Descripción
POST	/api/registro	Registra un nuevo usuario y devuelve token Bearer.
POST	/api/login	Autentica y devuelve token Bearer + datos de usuario.
POST	/api/logout	Revoca el token actual (requiere autenticación).
POST	/api/password-forgot	Solicita recuperación de contraseña al admin.
GET	/api/user	Devuelve el perfil del usuario autenticado.
PUT	/api/user/update	Actualiza el perfil del usuario autenticado.

4.5.3 Usuarios, Pedidos y Contabilidad

Método	Ruta	Descripción
POST	/api/pedidos	Crea un nuevo pedido de reparación (cliente autenticado).
GET	/api/mis-pedidos	Lista los pedidos del cliente autenticado.
GET	/api/pedidos/seguimiento/{codigo}	Consulta pública de estado por código.
GET	/api/admin/pedidos	Lista todos los pedidos con datos de cliente (admin).
PUT	/api/admin/pedidos/{id}/estado	Actualiza el estado de un pedido (admin).
GET	/api/admin/usuarios	Lista todos los usuarios registrados (admin).
PUT	/api/admin/usuarios/{id}/rol	Cambia el rol de un usuario (admin).
PUT	/api/admin/usuarios/{id}/password	Restablece la contraseña de un usuario (admin).
GET	/api/admin/contabilidad	Lista todos los registros contables (admin).
POST	/api/admin/contabilidad	Registra un nuevo asiento contable (admin).

5. Instalación y configuración básica

5.1 Prerrequisitos

Antes de instalar Josetech asegúrate de tener instalado el siguiente software:

- PHP 8.2 o superior con las extensiones: openssl, pdo, mbstring, tokenizer, xml, ctype, json.
- Composer (gestor de dependencias PHP) v2.x.
- Node.js v20.19.0 o superior (o v22.12.0+) con npm incluido.
- MySQL 8.x (recomendado para producción) o SQLite (para desarrollo local).
- Git para clonar el repositorio.

5.2 Configuración del Frontend

Desde la raíz del repositorio, ejecuta los siguientes comandos:

```
# 1. Acceder a la carpeta del frontend
cd frontend

# 2. Instalar dependencias de Node.js
npm install

# 3. Arrancar el servidor de desarrollo (HMR activado)
npm run dev

# 4. Generar build de produccion
npm run build
```

El servidor de desarrollo estará disponible por defecto en `http://localhost:5173`. Asegúrate de que la URL del backend en los archivos de stores y router coincida con tu entorno local (por defecto `http://127.0.0.1:8000`).

5.3 Configuración del Backend

Desde la raíz del repositorio, ejecuta los siguientes comandos:

```
# 1. Acceder a la carpeta del backend
cd backend

# 2. Instalar dependencias PHP
composer install

# 3. Copiar el fichero de entorno
cp .env.example .env
```

```
# 4. Generar la clave de aplicacion
php artisan key:generate

# 5. Configurar la conexion a la base de datos en .env
#   DB_CONNECTION=mysql
#   DB_HOST=127.0.0.1
#   DB_PORT=3306
#   DB_DATABASE=josetech
#   DB_USERNAME=root
#   DB_PASSWORD=tu_password

# 6. Ejecutar migraciones y seeders
php artisan migrate --seed

# 7. (Opcional) Configurar correo en .env
#   MAIL_MAILER=smtp
#   MAIL_HOST=smtp.gmail.com
#   MAIL_PORT=587
#   MAIL_USERNAME=tu@email.com
#   MAIL_PASSWORD=tu_app_password

# 8. Arrancar el servidor de desarrollo
php artisan serve
```

El backend estará disponible por defecto en `http://127.0.0.1:8000`. Para usar el modo de desarrollo completo con logs en tiempo real y cola de trabajos en un solo comando, puedes usar:

```
composer run dev
```

6. Conclusiones

El desarrollo de Josetech ha supuesto un recorrido completo por el ciclo de vida de una aplicación web moderna, desde el diseño de la base de datos hasta el despliegue de una SPA con API RESTful. El resultado es una plataforma funcional, coherente y escalable que resuelve un problema real: la digitalización de un taller de reparación de dispositivos electrónicos.

6.1 Dificultades

A lo largo del desarrollo se encontraron varios desafíos técnicos significativos:

Autenticación con Laravel Sanctum y Vue 3

La configuración de Sanctum para funcionar como API stateless con tokens Bearer (en lugar del modo cookie-based para SPAs) requirió comprender bien la diferencia entre los dos modos de uso. Fue necesario añadir el trait `HasApiTokens` al modelo `User` y configurar correctamente los headers `Accept: application/json` y `Authorization: Bearer {token}` en cada petición del frontend.

Protección de rutas por rol en el router de Vue

Implementar las guardas de navegación (`beforeEach`) que verifican el rol del usuario requirió hacer una petición asíncrona a la API antes de permitir la navegación, lo que introdujo complejidad para manejar el estado de carga y los posibles errores de red.

Gestión del estado global con Pinia

Sincronizar el estado del usuario autenticado (almacenado en `localStorage` como token) con el store de Pinia al recargar la página requirió implementar el método `verificarSesion()` que se llama al inicializar la aplicación para rehidratar el estado.

Integración de APIs externas (iFixit e Icecat)

Las APIs externas presentaron problemas de certificados SSL (solucionado con `withoutVerifying()`) y variaciones en la estructura de las respuestas JSON que requirieron manejo defensivo con el operador `??` para valores por defecto.

Escalabilidad del catálogo de dispositivos

La decisión de crear una vista específica por cada marca y tipo de dispositivo (más de 50 vistas) ofrece una experiencia personalizada pero implica mucho código repetitivo. Esto se identificó como una deuda técnica que podría resolverse con una vista dinámica parametrizada.

6.2 Mejoras futuras

Josetech tiene un buen punto de partida, pero hay varias áreas de mejora identificadas para versiones futuras:

- Vista dinámica de catálogo: reemplazar las 50+ vistas estáticas por una única vista `/catalogo/:tipo/:marca` que cargue los dispositivos dinámicamente desde la API, reduciendo drásticamente el código del frontend.
- Sistema de notificaciones en tiempo real: integrar Laravel Broadcasting con Pusher o WebSockets nativos para notificar al cliente cuando su pedido cambia de estado, sin necesidad de recargar la página.
- Panel de estadísticas: ampliar el módulo de contabilidad con gráficas de ingresos, reparaciones por marca y tasa de ocupación del taller usando una librería como Chart.js o Recharts.
- Subida de imágenes: implementar un sistema de carga de imágenes para dispositivos y pedidos (fotos del daño) usando Laravel Storage con Amazon S3 o almacenamiento local.
- Testing automatizado: ampliar la cobertura de tests con PHPUnit para los controladores del backend y Playwright para los flujos principales del frontend (login, solicitud de reparación, seguimiento).
- Recuperación de contraseña por email: implementar el flujo completo de reset de contraseña con tokens temporales usando el sistema nativo de Laravel (Password Broker) en lugar de la notificación manual al admin.
- Dockerización: crear un docker-compose.yml que levante el entorno completo (PHP-FPM + Nginx + MySQL + Node) con un único comando, facilitando el onboarding de nuevos desarrolladores.

7. Bibliografía

Documentación oficial y recursos de referencia utilizados durante el desarrollo del proyecto:

Frameworks y Lenguajes

- Laravel Documentation (v12.x). <https://laravel.com/docs/12.x>
- Vue.js 3 Official Documentation. <https://vuejs.org/guide/introduction>
- Vite Official Documentation. <https://vitejs.dev/guide/>
- Tailwind CSS Documentation. <https://tailwindcss.com/docs>
- PHP Manual (v8.2). <https://www.php.net/manual/en/>

Autenticación y Seguridad

- Laravel Sanctum Documentation. <https://laravel.com/docs/12.x/sanctum>
- Pinia - The Intuitive Store for Vue.js. <https://pinia.vuejs.org/>
- Vue Router Official Documentation. <https://router.vuejs.org/>

APIs Externas

- iFixit API Documentation (v2.0). <https://www.ifixit.com/api/2.0/doc>
- Icecat Open Catalog API. <https://icecat.us/developer/>

Herramientas y Utilidades

- Playwright Testing Framework. <https://playwright.dev/>
- Composer - PHP Dependency Manager. <https://getcomposer.org/>
- npm Documentation. <https://docs.npmjs.com/>

Recursos de Aprendizaje

- Laracasts - Laravel Video Tutorials. <https://laracasts.com/>
- Vue Mastery - Vue.js Courses. <https://www.vuemastery.com/>
- MDN Web Docs - Mozilla Developer Network. <https://developer.mozilla.org/>